ASSIGNMENT – 1

DATA STRUCTURES

Batch Code	CSA0305B
Name	Anbuselvan
Register Number	1972260038B

Assignment Topics

A. Differentiate between Data and Information with examples (5 marks)

B. Solve Fibonacci using recursion and dynamic programming; compare complexities (10 marks)

C. Describe applications of data structures in social networks (5 marks)

Submitted by: Anbuselvan

A. Differentiate Between Data and Information with Examples (5 Marks)

Data refers to raw facts, figures, or observations that have not yet been processed or given context. **Information** is data that has been organized, processed, or interpreted so that it becomes meaningful and useful.

Differences between Data and Information

Data	Information
Raw and unprocessed facts.	Processed and meaningful facts.
May not have a clear meaning by itself.	Provides context and helps understanding.
Used as input for processing.	Usually produced as the output of processing.
Example: 65, 72, 80, 91.	Example: The average mark is 77.
Example: Daily sales figures.	Example: A report showing total monthly sales.

Example: In a college, individual marks such as 70, 85, and 90 are data. When these marks are calculated to find the class average or identify the highest mark, the result is information that helps a teacher understand student performance.

Conclusion: Data is the raw material, while information is the meaningful result obtained after processing data.

B. Fibonacci Using Recursion and Dynamic Programming (10 Marks)

The Fibonacci sequence is a series in which each number is the sum of the two preceding numbers. It starts with 0 and 1: 0, 1, 1, 2, 3, 5, 8, 13, ... Mathematically, F(0) = 0, F(1) = 1, and F(n) = F(n–1) + F(n–2) for n ≥ 2.

1. Recursive Method

The recursive method solves the problem by calling the same function for the two previous Fibonacci values. It is simple to understand, but it repeats many calculations.

```
#include <stdio.h>

int fib(int n) {
if (n <= 1)
return n;
return fib(n - 1) + fib(n - 2);
}

int main(void) {
int n = 7;
for (int i = 0; i < n; i++)
printf("%d ", fib(i));
return 0;
}
```

Output: 0 1 1 2 3 5 8

Complexity: Time: O(2■) approximately, because many values are recalculated. Auxiliary space: O(n) for the maximum recursion call stack.

2. Dynamic Programming (Bottom-Up)

Dynamic programming stores results that have already been calculated. In the bottom-up method, the sequence is built from the base values 0 and 1. Each new value is calculated once.

```
#include <stdio.h>

int main(void) {
int n = 7;
int fib[100];

fib[0] = 0;
fib[1] = 1;

for (int i = 2; i < n; i++)
fib[i] = fib[i - 1] + fib[i - 2];

for (int i = 0; i < n; i++)
printf("%d ", fib[i]);

return 0;
}
```

Output: 0 1 1 2 3 5 8

Complexity: Time: O(n), because each Fibonacci value is calculated once. Space: O(n) for the array that stores the sequence.

3. Comparison of Complexities

Feature	Recursion	Dynamic Programming
Approach	Calls itself for smaller values.	Stores and reuses calculated values.
Time complexity	O(2■) approximately.	O(n).
Space complexity	O(n) call stack.	O(n) array.
Efficiency	Slow for large n due to repeated work.	Faster for large n because each value is computed once.

Conclusion: Recursion is useful for understanding the Fibonacci definition, but dynamic programming is more efficient because it avoids repeated calculations.

C. Applications of Data Structures in Social Networks (5 Marks)

Social networks such as Instagram, Facebook, and LinkedIn connect users and allow them to share posts, messages, photos, and other content. Data structures help these platforms store, organize, search, and process large amounts of information efficiently.

1. Graphs – Users are represented as vertices (nodes), and friendships or follows are represented as edges. Graphs help find mutual friends, connections, and possible new people to follow.

2. Hash tables – They can store user IDs, usernames, and profile references, allowing fast lookup of an account.

3. Trees – Trees and tree-like indexes can organize comments, replies, and content indexes for efficient searching.

4. Queues – Queues can manage tasks such as processing posts, notifications, and messages in the order they are received.

5. Arrays and linked lists – Arrays can store ordered lists of posts or user details, while linked structures can help represent changing collections of connections or items.

6. Heaps / priority queues – These can help prioritize notifications or rank content according to factors such as relevance and recency.

Example: When a user opens a social-network app, a graph can help identify friends-of-friends, a hash table can locate a profile quickly, and a priority queue can help order notifications.

Conclusion: Appropriate data structures make social networks faster, more scalable, and better at managing relationships and content.